

Programação Concorrente - Primeira Lista de Exercícios

Instituto de Computação - Prof. Gabriel P. Silva

Nome: Davi dos Santos Mattos DRE: 119133049

1. Em relação aos processos e threads, assinale as alternativas como verdadeiras ou falsas:
(0,5 ponto)

(V) Em um sistema com threads, se uma thread alterar o valor de uma variável global, essa alteração é visível para todas as outras threads do mesmo processo.

(F) Criar um processo via fork é mais rápido do que criar uma thread POSIX, pela ausência de mecanismos de sincronização no momento da criação do processo.

(V) A comunicação entre processos exige o uso de mecanismos como pipes, soquetes ou troca de mensagens, devido ao seu isolamento de memória.

(F) Para servidores de rede a criação de um processo independente para cada conexão é sempre a estratégia mais eficiente do que o uso de threads.

(V) O uso de processos é mais apropriado em cenários onde a segurança e a robustez são prioridades, pois o isolamento impede que um erro em um fluxo de execução corrompa a memória de outro.

2. Para cada cenário abaixo, indique qual(is) API(s) seria(m) mais apropriada(s) e justifique:
(0,5 ponto)

a. Paralelizar laços em um programa científico C++ que roda em um servidor com 64 núcleos

R: OpenMP, pois facilita a paralelização de códigos em C++, uma vez que OpenMP gerencia a criação e sincronização das threads automaticamente.

b. Acelerar multiplicação de matrizes usando GPU NVIDIA

R: CUDA, pois é desenvolvida pela própria NVIDIA para as suas GPUs

c. Simular um sistema climático global em um supercomputador com 1000 nós

R: MPI, pois é voltado para clusters e supercomputadores por causa da memória distribuída.

d. Processar imagens médicas em diferentes tipos de aceleradores (GPUs AMD, Intel, NVIDIA)

R: OpenACC, pois diferentemente da CUDA que é focada em GPUs da NVIDIA, a OpenACC é mais portátil.

e. Desenvolver uma simulação de dinâmica molecular de altíssimo desempenho para um cluster onde cada nó possui 32 núcleos de CPU e duas GPUs.

R: MPI e OpenACC. MPI para o cluster, CUDA para extrair o máximo de cada GPU dentro do nó

3. Com relação aos ambientes e paradigmas de programação concorrente e paralela, responda: (1,0 ponto)

a. Por que muitas aplicações de alto desempenho combinam MPI com OpenMP ou CUDA? Que vantagens essa combinação oferece?

R: Pois geralmente as APIs são focadas em um cenário em específico, e combina-las nos permite ganhar em desempenho e reduzir o custo computacional. Neste caso, o MPI realiza a comunicação entre os vários nós de um cluster (memória distribuída), enquanto o OpenMP e o CUDA exploram o paralelismo massivo dentro de cada nó (memória compartilhada ou GPUs)

b. O que são condições de corrida (race conditions)? Por que elas são problemáticas em programas concorrentes? Dê um exemplo hipotético.

R: Condição de corrida é quando duas operações de fluxos de execução distintos acessam um local de memória compartilhado de forma concorrente.

Em um cenário onde temos uma thread que **escreve** e outra que apenas **lê** o conteúdo, pode ocorrer da leitura ocorrer durante a escrita, por exemplo, supondo que temos um programa que indique o número do pedido pronto, uma thread 1 é responsável por informar qual pedido está pronto e outra (thread 2) é responsável por "avisar" qual pedido está pronto, é como se enquanto a T1 estivesse informando "Pedido do..." (pedido_pronto=pedido), a T2 informasse "Pedido 1 pronto" (valor do ultimo pedido)

c. Qual o impacto que o desbalanceamento de carga tem no desempenho final das aplicações paralelas? (0,5 ponto)

R: Impactam diretamente no speedup e eficiência do programa.

d. Um sistema operacional moderno gerencia a concorrência em múltiplos níveis. Identifique e explique pelo menos três níveis onde a concorrência aparece em um SO.

R: Interrupções, Processos, Threads, e o próprio SO.

e. Explique a diferença entre corrotinas (async/await) e threads tradicionais. Em que tipo de aplicação as corrotinas são particularmente vantajosas?

R: async/await permite que o fluxo de execução seja suspenso mas retorne depois, a thread tradicionais são diversos fluxos de execuções rodando em paralelo, onde compartilham o mesmo espaço de memória e recursos globais, diferente de async/await . async/await é vantajoso em casos de I/O onde a o fluxo de execução tem que esperar algum retorno.

4. Escreva um pequeno programa em C que utilize fork() para criar três processos filhos. Cada filho deve imprimir seu PID e o PID do pai, enquanto o pai deve esperar por todos antes de encerrar. (1,0 ponto)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    printf("Processo pai (PID: %d) iniciando...\n", getpid());

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            fprintf(stderr, "Erro no fork\n");
            return 1;
        }
        else if (pid == 0) {
            printf("Sou o filho (PID: %d)\n", getpid());
            printf("Processo Filho (%d) terminou.\n", getpid());
            exit(0); // filho termina aqui
        }
        // apenas o pai chega aqui
    }

    //printf("Sou o processo pai (PID: %d). Esperando os filhos...\n",
    getpid());
    // pai espera todos os filhos
    for (int i = 0; i < 3; i++) {
        wait(NULL);
    }
    printf("Pai terminou.\n");

    return 0;
}
```

5. No exemplo da soma de um vetor, a variável `somatotal` é compartilhada entre threads. Descreva por que a operação `somatotal = somatotal + somalocal` precisa de exclusão mútua, e as atualizações `soma_local` e o índice `i` do laço interno da thread não. (0,5 ponto)

R: Justamente por `soma_total` ser uma variável compartilhada entre todas as threads, enquanto `soma_local` só existe no escopo local daquela thread, evitando assim a corrida de dados.

6. Em que situação específica um programador precisaria configurar um mutex como `PTHREAD_MUTEX_RECURSIVE`? O que acontece se uma thread tentar travar duas vezes um mutex comum (não recursivo)? (0,5 ponto)

R: Quando uma thread que já possui `lock` precisa executar `lock` sem se auto bloquear. Neste caso a thread ficaria bloqueada para sempre (deadlock), uma vez que a própria thread é a única que pode liberar (`unlock`)

7. Com relação ao código de contagem de primos, responda: (0,5 ponto)

a. Na versão com divisão em blocos, explique por que a distribuição estática de trabalho pode gerar desequilíbrio entre as threads.

R: Pois nos intervalos finais, o cálculo de primalidade se torna mais complexo e custoso computacionalmente.

b. Na versão com saco de tarefas, o índice global `i_global` é protegido por um mutex. Explique por que essa proteção é necessária e qual trecho do código constitui a seção crítica.

R: `i_global` é uma variável compartilhada por todas as threads, então na seção `if ( i_global >= N ){..} i_local = i_global ; i_global += 2;` poderia ocorrer corrida de dados sem o mutex.

8. Por que a versão do cálculo de número primos com saco de tarefas apresenta melhor balanceamento de carga em comparação com a divisão em blocos? Por que a sua eficiência melhorou, mesmo usando mutex? (0,5 ponto)

R: Há uma distribuição de cargas automática fazendo com que as threads ignorem número pares, e as que terminarem o cálculo mais rápido, pegam mais tarefas, enquanto as outras estão ocupada. O uso do mutex é mínimo ocorrendo somente na etapa de alocação do índice.

9. Quais são as quatro formas principais de sincronização citadas para programas de memória compartilhada? Como são mapeadas em bibliotecas da linguagem C? (0,5 ponto)

R: Exclusão mútua (`pthread_mutex_t`), Sincronização por condição (`pthread_cond_t`), barreira (`pthread_barrier_t`) e semáforos (`sem_t`).

10. Com relação ao uso de sincronização por variáveis condicionais, responda: (0,6 ponto)

a. Qual a diferença funcional entre as operações SIGNAL e BROADCAST em uma variável de condição.

R: `pthread_cond_signal` envia um sinal para desbloquear uma única thread aleatoriamente, enquanto `pthread_cond_broadcast` envia sinais para desbloquear todas as threads.

b. Por que uma variável de condição deve ser sempre usada em conjunto com um mutex?

R: Para proteger a verificação da lógica de condição de múltiplos acessos indevidos.

c. O que acontece com o lock (mutex) quando uma thread executa a operação `pthread_cond_wait()` e o que ocorre quando ela retoma a execução?

R: `lock(mutex)` indica o início de uma seção crítica, impedindo que outras threads executem a seção crítica, após executar o `wait`, o lock é automaticamente liberado para permitir que outras threads entrem na seção crítica e alterem a condição, quando uma thread retorna de `wait`, o lock é readquirido.

11. Com relação à sincronização com uso de barreiras, responda: (0,4 ponto)

a. Defina o conceito de sincronização por barreira e explique por que ela é comparada a um "ponto de encontro". (0,5 ponto)

R: Sincronização de barreira é um mecanismo que define que todas as threads executem o código até determinado ponto e esperem todas as threads antes que elas sigam para o próximo passo. Esse conceito é comparado a um ponto de encontro pois cada thread pode estar executando uma coisa diferente ou em pontos diferentes, e em algum momento elas devem parar e esperar as outras.

b. Quais são as rotinas da biblioteca Pthreads utilizadas para inicializar, esperar e destruir uma barreira? (0,5 ponto)

R: Iniciar - `pthread_barrier_init`

Esperar - `pthread_barrier_wait`

Destruir - `pthread_barrier_destroy`

12. Com relação aos semáforos, responda: (1,0 ponto)

a. Qual a diferença entre semáforo binário e um mutex?

R: Com `mutex` a mesma thread que trava deve destravar, enquanto com semáforo binário uma thread pode ser destravada por diferentes threads

b. Qual a diferença entre um semáforo de contagem e um semáforo binário?

R: Semáforo binário assume valores 0 ou 1, enquanto semáforo de contagem assume valores de 0 a N.

c. Em que situações um semáforo é mais útil que um mutex?

R: Quando precisamos gerenciar o acesso simultâneo de mais de uma thread em algum recurso limitado.

d. Quais são as duas operações atômicas utilizadas para acessá-los?

R: wait() e post()

e. Quais as principais limitações de um semáforo?

R: A inversão de prioridade e o Alto risco de deadlock.

13. Explique a diferença de escopo entre um semáforo inicializado com sem_init() e um aberto com sem_open(). Qual deles é mais adequado para sincronizar dois processos independentes que não compartilham memória? (0,5 ponto)

R: sem_init() é um semáforo não nomeado criado em memória, enquanto sem_open() é associado a um nome global e pode ser acessado por diferentes processos, mesmo que não compartilhem a memória. Tendo isso em vista sem_open() é mais adequado.

14. Implementar uma rotina para emular o comportamento de uma barreira para 3 threads utilizando apenas um Mutex e uma Variável de Condição (sem usar pthread_barrier_t). A última thread a chegar deve dar o sinal para todas as outras. (1,0 ponto)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 3

pthread_mutex_t mutex;
pthread_cond_t cond;
int chegou = 0;

void *task(void *arg)
{
    int id = (int)arg;

    pthread_mutex_lock(&mutex);

    chegou++;

    printf("Thread %d terminou passo 1\n", id);
```

```

    if (chegou == NUM_THREADS)
    {
        pthread_cond_broadcast(&cond);
    }
    else{
        pthread_cond_wait(&cond, &mutex);
    }
    pthread_mutex_unlock(&mutex);
    printf("Thread %d terminou\n", id);
    return NULL;
}

int main()
{
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL, task, (void *)i))
        {
            fprintf(stderr, "Error\n");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);
    return 0;
}

```

15. Estenda o exemplo de produtor-consumidor para usar um array (buffer) de 5 posições. O produtor só pode produzir se houver espaço, e o consumidor só pode consumir se o buffer não estiver vazio. Utilize variáveis de condição para gerenciar esses estados. (1,0 ponto)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

```

```

#define BUFFER_SIZE 5
#define NUM_ITEMS 10

int buffer[BUFFER_SIZE];
pthread_mutex_t mutex;
pthread_cond_t cond_cheio, cond_vazio;
int cheio = 0;

void *produtor(void *arg){

    static int in = 0;

    printf("Iniciando Produção\n");
    for (int i = 0; i < NUM_ITEMS; i++)
    {
        pthread_mutex_lock(&mutex);
        while (cheio == BUFFER_SIZE) {
            printf("P - Buffer cheio. Aguardando...\n");
            pthread_cond_wait(&cond_vazio, &mutex);
        }
        buffer[in] = i;
        printf("Produzido: %d\n", buffer[in]);
        in = (in + 1) % BUFFER_SIZE;
        cheio++;
        pthread_cond_signal(&cond_cheio);
        pthread_mutex_unlock(&mutex);
    }

    pthread_mutex_lock(&mutex);
    while (cheio == BUFFER_SIZE) pthread_cond_wait(&cond_vazio, &mutex);
    buffer[in] = -1;
    cheio++;
    printf("Producao finalizada\n");
    pthread_cond_signal(&cond_cheio);
    pthread_mutex_unlock(&mutex);
    pthread_exit(NULL);
}

void *consumidor(void *arg){

    static int out = 0;
    int item;

    while (1)
    {
        pthread_mutex_lock(&mutex);

```



```

        while (cheio == 0) {
            printf("C - Buffer vazio. Aguardando...\n");
            pthread_cond_wait(&cond_cheio, &mutex);
        }
        item = buffer[out];
        if (item == -1)
        {
            pthread_mutex_unlock(&mutex);
            break;
        }

        printf("Consumido: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;
        cheio--;
        pthread_cond_signal(&cond_vazio);
        pthread_mutex_unlock(&mutex);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond_cheio, NULL);
    pthread_cond_init(&cond_vazio, NULL);

    pthread_t prod, cons;

    pthread_create(&prod, NULL, produtor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond_cheio);
    pthread_cond_destroy(&cond_vazio);

    return 0;
}

```